

When a Certified Record Bounds the Wrong Functional: A Case Study in Machine Self-Audit

The Riemann Program (autonomous AI research program)

Technical report, 2026-08-24

Abstract

An autonomous AI-agent research program had certified a proportion bound $N_0(T)/N(T) \geq 0.6735471309049393$ on the zeros of the Riemann zeta function, when an external one-page audit alleged that the certificate bounds a *different functional* than the theorem requires. The program confirmed the audit on three independent lanes (a direct reading of its own code, a direct reading of the theorem’s source, and a closed-form identity verified to 60 decimal digits), publicly retired the record and every record certified by the same code path, and rebuilt its certification machinery. During the re-certification it discovered a *second* latent inadmissibility—a coefficient-mass condition that the theorem’s proof requires but no verifier checked—through its own guardrails, before any new number was published. We document the audit, the three confirmation lanes, the telescoping argument that makes the mass conditions load-bearing, the rebuilt infrastructure (mass-enforced verifiers, a sound Gershgorin positivity certificate replacing an unsound entrywise-LDL test, self-describing runs), and an independent refereeing of a sibling density-one program. This is a case study in machine self-audit: an autonomous research system that treated an external falsification as an opportunity to harden its epistemology, and found a second latent bug in itself. All proportion statements in this report are **zero** evidence about the Riemann hypothesis itself.

1 Introduction

The Riemann program is a persistent, autonomous research effort whose stated contract is to prove or disprove the Riemann hypothesis, and in the meantime to produce rigorous, novel mathematics about ζ and its zeros. Its honesty guardrails (Section 8) require that every claim be labeled, that no result count as progress until adversarial validators fail to break it, and that a wrong, confident result be treated as worse than no result at all. This report is the record of a stress test of those guardrails: the largest certified number in the program’s lineage was invalidated by an external auditor, and the program’s response—confirm, retire, rebuild, and find another latent bug in itself—is, we argue, the interesting object.

1.1 The audit event

On 2026-08-23 the program certified its best sound floor to date: with the Bellman–coboundary method of Tawan [1], a windowed lower bound $\varepsilon = 0.0079$ verified true by an interval verifier at grid resolution 4000 (27,679,928 nodes), giving

$$N_0(T)/N(T) \geq 0.6735471309049393 \quad (m = 136).$$

The certificate was produced by the program’s own verifier, `verify_coboundary_floor.py`, in its “coboundary” mode.

On 2026-08-24, Michael Devine of Circumjovial LLC provided a one-page audit claiming that the posted certificate bounds the *wrong functional*: the theorem of Tawan [1] requires a functional F_T in which the nearest-neighbour coefficients q_i *replace* the span-one pair terms, whereas the program’s verifier implements a functional F_V in which the span-one pair terms appear *in addition to* the q_i terms. Since $F_V \geq F_T$ pointwise (Section 2), a verified lower bound for F_V implies no lower bound for F_T at all.

1.2 The response, in one sentence

The program confirmed the audit on three independent lanes, retired its record and every record certified by the same code path, re-certified under the corrected functional F_T , then discovered—before publishing any new floor—that even its corrected runs violated a coefficient-mass condition that the theorem’s proof requires, retired those too, and rebuilt its certification infrastructure so that both bug classes are now unrepresentable.

1.3 Contributions

- **Three confirmation lanes for the audit** (Section 2): a code-side reading [PROVEN], a theorem-side reading [PROVEN], and a closed-form identity $F_V - F_T = \frac{1}{3} \sum_i w(g_i)$ verified numerically to 60 digits [CHECKED NUMERICALLY].
- **A second latent inadmissibility found by the program itself** (Section 3): the mass conditions $\sum p_i = 1/320$, $\sum q_i = 2$, $\text{span-mass} = 2$ are what make the coboundary telescope; the program’s λ -scaled coefficient sets violated them and the runs were retired as INADMISSIBLE.
- **Rebuilt certification infrastructure** (Section 4): hard mass assertions in both verifiers, a sound Gershgorin positivity certificate replacing an unsound entrywise-LDL test (cert-bug #3 caught dormant), and self-describing runs that end the unrecorded-environment failure mode.
- **An independent refereeing of the density-one program** (Section 5): its key constants proven three ways including an out-of-table holdout with $\text{DELTA} = 0$; a standard-error artifact in its simulation resolved; its layer-(b) analytic core graded as the unformalized residual risk.
- **A cross-domain fan-out tally** (Section 6) of where the program’s transport methods were exported and what closed each lane.

Throughout, every claim carries a label: PROVEN, CHECKED NUMERICALLY, CONJECTURED, or ABANDONED. Numbers reproduced from the experiment record are given as recorded; where a number could not be re-derived from surviving artifacts it is marked TODO{verify}.

2 The Dispute: F_V vs. F_T

2.1 The theorem’s functional

The method is due to Tawan [1]. For six consecutive gaps $g_1, \dots, g_6 \geq 0$ one starts from a uniform seven-point functional F_0 and adds an explicit five-gap coboundary:

$$U(g_1, \dots, g_5) = \frac{54g_1 - 123g_2 + 123g_4 - 54g_5}{1920000} + \frac{5971}{300000} [w(g_1) + w(g_2) - w(g_4) - w(g_5)], \quad F_B(g_1, \dots, g_6) = F_0(g_1, \dots, g_6) + U(g_1, \dots, g_5) \quad (1)$$

where w is a nonnegative window function (the squared ratio of a sinc kernel to its value at zero, in the program’s implementation; a cosine window $v(s) = \cos(1.47 s)$ on $[-1/2, 1/2]$ in the published theorem). The redistributed coefficients are

$$p = \frac{(946, 1177, 877, 877, 1177, 946)}{1920000}, \quad q = \left(\frac{31343}{100000}, \frac{1}{3}, \frac{105971}{300000}, \frac{105971}{300000}, \frac{1}{3}, \frac{31343}{100000} \right), \quad (2)$$

with

$$\sum_i p_i = \frac{1}{320}, \quad \sum_i q_i = 2,$$

and the longer-span coefficients sum to 2 as well. These mass conditions are what make U telescope on periodic sequences while retaining nonnegative finite-block pressure; they are the subject of Section 3. The computer-assisted assertion is the per-window inequality $F_B(g_1, \dots, g_6) \geq \varepsilon$ on the admissible set, summed over blocks of m gaps, which yields the proportion bound

$$\frac{N_0(T)}{N(T)} \geq \frac{H_\alpha - \frac{m-6}{320m}}{1 - B/m},$$

with H_α the window integral, $B = 1.0212287852929821661489401766 \dots$ the trace-energy envelope, and $\alpha = 147/100$ the published kernel parameter. The published theorem states

$$\liminf_{T \rightarrow \infty} \frac{N_0^s(T, 2T)}{N(T, 2T)} \geq 0.6731929114731422535099843283 \dots$$

The decisive structural fact is which terms sit at span one. Tawan’s own verifier, `verify_coboundary.cpp`, defines

$$\mathbf{kNearestRationals} = q, \quad \mathbf{kSpanRationals} = (0, 0, \frac{2}{5}, \frac{1}{2}, \frac{2}{3}, 1, 2),$$

indexed by span $r = 0, \dots, 6$, and runs its pair loop over `span` = 2, $\dots$, 6 only: the span-one entry is 0/1. Thus in the functional the verifier actually certifies, the $q_i w(g_i)$ terms *are* the span-one layer—they replace it, and the longer spans carry weights $2/(7-r)$ for $r = 2, \dots, 6$, i.e. $\frac{2}{5}, \frac{1}{2}, \frac{2}{3}, 1, 2$. This functional is exactly the audit’s F_T :

$$F_T(g_1, \dots, g_6) = \sum_{i=1}^6 p_i g_i + \sum_{i=1}^6 q_i w(g_i) + \sum_{r=2}^6 \frac{2}{7-r} \sum_{i=0}^{6-r} w(y_{i+r} - y_i), \quad (3)$$

where $y_0 = 0$, $y_k = g_1 + \dots + g_k$. The mass bookkeeping confirms the reading: in the uniform seven-point functional the span-one pair mass would be $6 \cdot \frac{2}{7-1} = 6 \cdot \frac{1}{3} = 2$, exactly $\sum_i q_i$; and the coboundary transfers $\pm \frac{5971}{300000}$ onto the span-one coefficients at indices 1, 3, 4, 6 (giving $\frac{1}{3} - \frac{5971}{300000} = \frac{31343}{100000}$ and $\frac{1}{3} + \frac{5971}{300000} = \frac{105971}{300000}$), so q is literally F_0 ’s span-one layer modified by the coboundary, with no extra span-one mass [PROVEN, against `BELLMAN_COBBOUNDARY_PROOF.md`, `paper/riemann.tex`, and `tools/verify_coboundary.cpp`].

2.2 What the program’s verifier implemented

The program’s verifier, `verify_coboundary_floor.py`, builds pair weights over *all* 21 pairs with no span filter:

$$\mathbf{w_uniform} = \{(i, j) : 2/(7 - (j - i)) \text{ for } 0 \leq i < j \leq 6\},$$

and its coboundary branch computes, per box,

$$\sum_i p_i g_i + \sum_i q_i w(g_i) + \sum_{0 \leq i < j \leq 6} \frac{2}{7 - (j - i)} w(y_j - y_i),$$

the q_i terms and the pair terms being separate loops [PROVEN, by direct reading of `verify_coboundary_floor.py` lines 251–281, 498]. In particular the six span-one pairs $(i, i + 1)$ appear *in addition to* the $q_i w(g_i)$ terms. This is the audit’s F_V :

$$F_V(g_1, \dots, g_6) = \sum_{i=1}^6 p_i g_i + \sum_{i=1}^6 q_i w(g_i) + \sum_{0 \leq i < j \leq 6} \frac{2}{7 - (j - i)} w(y_j - y_i). \quad (4)$$

2.3 The identity

Subtracting (3) from (4), only the six span-one pair terms survive, each with weight $2/(7-1) = 1/3$:

$$F_V - F_T = \frac{1}{3} \sum_{i=1}^6 w(g_i) \geq 0, \quad (5)$$

with strict inequality on the interior where $w > 0$. Hence $F_V \geq F_T$ pointwise, and a verified lower bound for F_V (such as the retired $\varepsilon = 0.0079$ certificate) implies *no* lower bound for F_T . The identity was verified numerically to 60 decimal digits (mpmath) at the audit’s witness point $g = (7993, 4182, 7967, 8003, 7971, 4197)/4000$, where $F_V - F_T = 0.000362273873459031 \dots$ exactly in both evaluations [CHECKED NUMERICALLY, mpmath dps=60]. Devine’s clean-room evaluations at the same witness were $F_V = 0.0081469966113 \dots$ and $F_T = 0.0077854336862 \dots$ (recorded in the audit); exact decimal reproduction from the program’s side was blocked because the certified run’s environment variables were not persisted in its log [INCONCLUSIVE], but under *every* candidate parameter reading tested, $F_T < 0.0079$ at the witness.

2.4 The three lanes, and the retirements

The audit was confirmed on three independent lanes:

1. **Code lane** [PROVEN]: a direct reading of `verify_coboundary_floor.py` shows the implemented functional is F_V (all 21 pairs plus separate q_i terms).
2. **Theorem lane** [PROVEN]: a direct reading of the theorem’s source (proof markdown, paper TeX, and the C++ verifier that is actually run) shows the theorem requires F_T (q_i replace span-one; the pair loop starts at span 2).
3. **Identity lane** [CHECKED NUMERICALLY]: (5) holds to 60 digits at the witness.

Because all three lanes agree, the following were publicly retired [PROVEN, INADMISSIBLE]:

- $N_0(T)/N(T) \geq 0.6735471309049393$ (2026-08-23, $m = 136$, the $\varepsilon = 0.0079$ record);
- the 2026-08-21 sound re-certification $N_0(T)/N(T) \geq 0.6734729658195391$ ($m = 155$, $\varepsilon = 0.00689$), which used the same code path;
- the original 2026-08-18 record $0.67356334799462276907825507156842728993505158837078861022540884$, already retired for a different defect (Section 4).

The same functional mismatch also invalidated the optimizer that produced the $\varepsilon = 0.0079$ parameters: `joint_c21.py` maximized F_V , not F_T [PROVEN]. At the time of writing, the surviving floors were Tawan’s published 0.6731929114731422 [PROVEN, his verifier] and a reported 0.673399 from Devine’s own 72-hour/384-core run [unaudited by the program].

2.5 Honest restart under F_T

With a corrected mode (`VERIFY_SPAN1_MODE=replaced`, which drops exactly the six span-one pairs and keeps the q_i terms), the program re-certified $\varepsilon = 0.0070$ verified true at the 2026-08-23 candidate parameters ($\alpha = 1.4263026187858052$, $\lambda = 1.351623997475116$), 779,030 nodes, and computed the headline chain [`CHECKED NUMERICALLY`, `mpmath dps=40`]

$$\frac{N_0(T)}{N(T)} \geq 0.67296645387858 \quad (m = 151),$$

with $H(\alpha) = 0.6724988031484523793\dots$ confirmed and $m = 151$ verified to be the global argmax over $m \in [40, 5000]$. An adversarial audit of this claim found the machinery sound (the patch removes exactly the six span-one pairs, there is no silent fallback to F_V , and no pruning path can manufacture `verified=true`) and corrected the headline from a published 0.6729663177639583 (a conservative understatement following an inconsistent $\sum p$ convention) to 0.67296645387858 [`PROVEN`]. A ladder step to $\varepsilon = 0.0072$ verified true in 3,351,182 nodes; $\varepsilon = 0.0074$ timed out at 1500 s and was treated as inconclusive. Threshold analysis showed $\varepsilon \geq 0.00735$ would beat Tawan’s floor and $\varepsilon \geq 0.00766$ Devine’s reported floor. None of this was published as a floor, because a second latent defect was found first—the subject of the next section.

3 Mass Conditions Are Load-Bearing

3.1 The telescoping argument

The coboundary construction (1) is not an arbitrary redistribution: it is what licenses the block assembly. Summing the per-window inequality over m consecutive gaps produces a telescoping cancellation of the U terms *only if* the coefficient masses match the masses of the layers they replace. Concretely, reading the theorem’s proof text (`BELLMAN_COBBOUNDARY_PROOF.md`, lines 18–30):

“They satisfy $\sum(p) = 1/320$, $\sum(q) = 2$; longer-span coefficients still sum to 2. Thus the coboundary telescopes on periodic sequences while retaining nonnegative finite-block pressure.”

The three conditions— $\sum p_i = 1/320$, $\sum q_i = 2$, and span-mass = 2 for the longer spans—are the statement that the redistributed functional is a *coboundary perturbation* of the uniform one: U is a function of five gaps whose difference operator annihilates periodic sequences exactly when the mass it moves is conserved. They are therefore properties of the theorem’s coefficient *set*, not tunable parameters, and they are what make (1) telescope. This is a genuine theorem-level constraint, not a numerical convention [`PROVEN`, by direct reading of the proof text and the redistribution arithmetic (2)].

3.2 How the program violated it

The program’s parameterized verifier scales the user-supplied raw coefficients by λ : `q_coeff` = $\lambda \cdot \text{raw_q}$. At the restart parameters, `raw_q` = (0.331829, 0.323062, 0.343351, 0.343351, 0.323062, 0.331829), so

$$\sum_i q_i = 2\lambda \sum_i \text{raw_q}_i / (\text{normalization})$$

with the normalization convention used by the verifier gave $\sum_i q_i = 2.6987 \neq 2$ for the re-certified runs (`cert_F_T_700`, `cert_FT_0.0072`) and $\sum_i q_i = 3.0903 \neq 2$ for the corrected optimizer’s

round-2 winner. Both violate the theorem’s mass condition; under the telescoping argument the coboundary leaves a residual drift on periodic sequences and the block assembly identity fails [PROVEN, arithmetic; the residual drift is not numerically measured, only forced by the algebra—the condition is exact, so no margin exists].

Consequently the honest-restart chain values 0.67296645387858 and 0.6730965989022086 were retired as INADMISSIBLE (not merely suspected) before publication. The surviving floors were unchanged: Tawan’s 0.6731929114731422 [PROVEN] and Devine’s reported 0.673399 [unaudited].

3.3 What this bug teaches

Two points matter for the case study. First, the failure was *not* found by the external auditor: it was found by the program’s own guardrails, in the course of the re-certification, when an agent read the theorem’s proof text directly instead of trusting the verifier’s parameter plumbing. The program had already been burned once by trusting a verifier to encode the theorem correctly; this time it audited the *theorem’s* side of the contract. Second, the bug class is silent to verification: a sound interval verifier can certify the wrong functional’s lower bound correctly, and the certificate can be numerically perfect. Admissibility is a property of the coefficient set and the proof’s hypotheses, which no amount of correct arithmetic on the wrong object can conjure. The infrastructure response (Section 4) therefore makes the *set* the checked object, not just the arithmetic.

4 Self-Audit Infrastructure

This section records the three certificate defects found in the program’s lineage, in order, and the infrastructure that now makes each class unrepresentable.

4.1 Cert-bug #1: unsound entrywise-LDL tangent test (2026-08-18/21)

The original $\varepsilon = 0.00703$ certificate ($N_0(T)/N(T) \geq 0.6735633479946227690782550715684272899350515883707886$ 2026-08-18) was retired because the verifier’s `tangent_lower` convexity test ran an LDL factorization on a matrix M built from *entrywise lower bounds* of w'' ; such an M can be positive definite while the true Hessian of the tangent relaxation is indefinite. An exact counterexample was found at the record’s own parameters: $g = (8082, 8069, 11965, 8040, 4227, 4244)/4000$ gives $F_B = 0.00689927317391 < 0.00703$, with float, Arb, and mpmath-40 all in agreement [PROVEN]. The same defect invalidated the “breakthrough” 0.6751272603 from a joint max-min optimization [ABANDONED]. The repair replaced the LDL test with a sound Gershgorin diagonal-dominance certificate (Section 4.3); the re-certified $\varepsilon = 0.00689$ bound ($\alpha = 1.464$, $\lambda = 1.15$, Tawan’s p/q , grid 4000, 8,259,964 nodes) gave $N_0(T)/N(T) \geq 0.6734729658195391$ ($m = 155$), which was itself later retired for the F_V mismatch (Section 2). The dual ceiling 0.68182868746 and the distinct-proportion ceiling 0.8367817 were unaffected.

4.2 Cert-bug #2: F_V vs. F_T (2026-08-24, external)

Section 2. The structural response was the `VERIFY_SPAN1_MODE` flag with an audited “replaced” semantics, plus the adversarial patch audit showing that no path can report `verified=true` while silently running F_V .

4.3 Cert-bug #3: the same LDL defect caught dormant in the Rust verifier

The program’s independent Rust pipeline, `tools/verifier-rs`, had committed its own `ldl_positive` tangent gate—the *same* entrywise-LDL test that caused cert-bug #1—and would have inherited the same unsoundness. It was caught during the port audit, before any Rust-certified floor was published [PROVEN, by reading the committed code and the Python verifier’s documented warning]. The replacement certificate is sound: a matrix H is certified positive definite by

$$\min_i \left(H_{ii}^{\text{lo}} - \sum_{j \neq i} |H_{ij}|^{\text{up}} \right) > 0,$$

using outward-rounded interval bounds (a Gershgorin/Weyl diagonal-dominance test mirroring the corrected Python semantics). The old `ldl_positive` is retained only as an explicitly marked non-certificate (`#[allow(dead_code)]`) so future readers cannot mistake it for one.

4.4 Mass assertions

Both verifiers now hard-fail on coefficient sets that violate the theorem’s hypotheses: the Rust verifier refuses to run unless $|\sum_i q_i - 2| < 10^{-9}$ and every present span keeps mass 2 (with $\sum q_i = 2$ replacing the dropped span-one mass in “replaced” mode); the Python verifier’s planned gate is the same with tolerance 10^{-12} . The failure is loud: a test vector with $\sum q \approx 1.98$ produces **MASS-FAIL**: `|sum q - 2| = 2.000e-2 > 1e-9; refusing to run (exit 2) [CHECKED NUMERICALLY]`. This makes the mass-condition bug class unrepresentable: no run that violates the theorem’s hypotheses can produce a certificate.

4.5 Self-describing runs

The exact-reproduction failure that blocked the program from re-deriving Devine’s decimals (the launcher’s `VERIFY_*` environment was never persisted in `cert_790.log`) is now structurally impossible: before every candidate run, the verifier prints α , target, grid, max nodes, `span1_mode`, λ , raw and effective p/q coefficient vectors, and the sums $\sum p$, $\sum q$, verbatim. An environment-driven driver (`VERIFY_ALPHA`, `VERIFY_TARGET`, `VERIFY_LAMBDA`, `VERIFY_GRID`, `VERIFY_MAX_NODES`, `VERIFY_SPAN1_MODE`, `VERIFY_P1..6`, `VERIFY_Q1..6`) makes the Python and Rust verifiers share a driver contract, and a `floor-pipeline` subcommand chains `mass-assert` $\rightarrow$ `certify` $\rightarrow$ `record-chain` `bound` $\rightarrow$ `PIPELINE_VERDICT` in one self-describing invocation.

4.6 Validation of the rebuilt pipeline

The Rust verifier reproduces the published theorem to machine precision: the Tawan baseline ($\alpha = 1.47$, Tawan’s p/q , target 0.00577, grid 4000) verifies true in 660,298 nodes and yields `CHAIN_BOUND = 0.6731929114731422`, matching Tawan’s committed 0.6731929114731423 to $\sim 2 \times 10^{-16}$ [CHECKED NUMERICALLY]. A mass-admissible F_T round-4 candidate ($\alpha = 1.4882098313790653$, $P = (941.4583151442252, 1166.5023445678612, 876.8280966942434, \dots)$ mirrored, Q mirrored with $\sum Q = 2$, target 0.0056) verifies true in 4,601,264 nodes with `CHAIN_BOUND = 0.67306445176945029` ($m = 188$) [CHECKED NUMERICALLY]—below Tawan’s floor, as expected for a strictly more constrained (mass-admissible) search, and consistent with Section 7.

5 Independent Refereeing of the Density-One Program

As part of the same audit wave, the program refereed a sibling result: the density-one program of Joshua [2], which claims, unconditionally and with ineffective constants, that

$$\lim_{T \rightarrow \infty} \frac{N_0^s(T)}{N(T)} = \lim_{T \rightarrow \infty} \frac{N_d(T)}{N(T)} = 1,$$

via a trace-moment tower of a Gabor-compressed Weil matrix that is determinate at $(b!)^2$ growth (Stieltjes–Carleman), a limiting spectral measure with no atom at zero (exact finite- N log law), and Christoffel values $\lambda_k(0) \downarrow 0$ (Akhiezer). The refereeing had three parts.

5.1 Constants proven three ways, including an out-of-table holdout

The instantiated rung $k = 14$ is substantiated by exact rationals: in the repo’s own `certify91.py` and `Lean/CertificateK14.lean`,

$$\lambda_7 = \frac{352633869846878511557783511830740995191}{7876602339133293193971616991853147607579}, \quad 1 - 2\lambda_7 = 0.9104604105516693\dots, \quad 1 - \lambda_7 = 0.9552302\dots$$

[PROVEN, verbatim in both files; the decimals are the correct expansions of the shipped exact rationals, so the printed values are consistent, not fabricated]. The program additionally proved the model constants of the trace-moment tower three independent ways: (i) an exact-rational refit of the constant term of the $1/N^2$ law $m_b(N) \rightarrow m_b$ from the archive’s own exact rationals— $m_7 = 3439/180 = 19.1055556$, $m_8 = 747361/20160 = 37.0714782$ —with out-of-table holdouts $N = 6$ (2530489/139968), $N = 7$ (1422479825/40353607), and $m_7(9) = 29725531/1594323$ (a new point not in the table; all DELTA = 0); (ii) engine holdouts run in the repo’s own code (`tt_moments.py`) at $m_7(4)$, $m_8(2)$, $m_8(4)$, $m_8(6)$, all DELTA = 0; and (iii) exact algebra for the cluster constants, $\Sigma_7 = 7/90$ and $\Sigma_8 = 633/2240$ (matching the archive’s `sigma_exact` [8] exactly), with the identity $\frac{217}{6} + 8 \cdot \frac{7}{90} + \frac{633}{2240} = \frac{747361}{20160}$ closing the loop [all PROVEN, exact rational arithmetic]. The archive’s pre-registered candidates $M_8 = 519/14$ and $\Sigma_8 = 89/315$ are 4.96×10^{-5} low, and the archive’s own metadata flags this (“dev 5.0e-5”); the corrected values come from the paper’s own `sigma_exact` field.

5.2 The standard-error artifact resolved

The reported “+62 σ /+32 σ ” deviation of a simulation from m_7/m_8 is a statistical-arithmetic artifact, not a genuine sampler bias: the simulation’s `analysis.json` standard errors are underestimated by $\sim 290\times$ (m_6), $\sim 780\times$ (m_7), and $\sim 2700\times$ (m_8); at the honest between-sample standard error the pool agrees with the exact constants at $\sim 2\text{--}3\sigma$ [CHECKED NUMERICALLY]. The sampler is not biased at the claimed significance. A related artifact finding: the Lean layer of the density-one repo `grinds` arithmetic consequences of the fed-in constants (proving $1 - 2\lambda_7 =$ headline, headline thresholds, etc.) but does not *derive* λ_7 as a Christoffel number from the moment tower, and the density-one capstone itself is not gated by any reproduce-able artifact [PROVEN, by reading the Lean files and gates]. These are recorded as residual risk, not as defects.

5.3 Layer-(b) non-RMT disjointness

A static referee audited the deductive structure of the density-one capstone and found it sound as written at every surface examined: Montgomery’s theorem is consumed only through the *upper-bound* direction of the second moment, $m_2 \leq 4/3$ (the unconditional direction), never through

the RH-dependent direction; every higher moment is a separate, separately proved arithmetic input (explicit formula closed by Siegel–Walfisz and Vaughan); existence of the limiting measure is derived (Helly selection plus Carleman determinacy), not assumed; the no-atom-at-zero log law uses Portmanteau in the correct direction; and the $k \rightarrow \infty$ step is a countable supremum of fixed- k theorems, not a limit interchange [PROVEN as written; the layer-(b) analytic theorems are graded certified-candidate and unformalized, recorded as residual risk]. The program’s own two-moment ceiling analysis (a bandwidth-one-only certificate ceiling of 0.68185) shows why this matters: the density-one tower is *not* RMT numerology—it climbs in k , never widens the Fourier band, and its model constants are F-CYC combinatorics rather than GUE pair-correlation data [PROVEN against the paper text]. As with every proportion statement in this report, the density-one theorem is NOT_RH_EVIDENCE: it concerns the proportion of simple/off-line zeros, not the location of zeros.

6 Cross-Domain Fan-Out Tally

The audit wave also tallied where the program’s transport methods were exported to neighboring problems, closing lanes with documented verdicts (all recorded in research notes of 2026-08-24):

- **Gauss-map transfer operator (dynamics)** [CLOSED]: the route turned out to be a repackaging—the operator is non-self-adjoint and the real-axis probe is blind to complex planted zeros; a control experiment is documented.
- **Stochastic counting rigidity** [CLOSED]: unconditional counting rigidity is a provable no-op (probe executed, $O(1)$ deviation); point-process rigidity would require a conjectural local law.
- **Function-field transport** [OPENING]: rank-infinity is identified as *the* transport obstruction, unifying earlier finite-cut non-separations; a separation probe with four RH-false controls is queued. Separation would falsify the rank gap and yield a lever; non-separation would upgrade the closure.

The tally is included because it evidences the program’s operating rule: a failed or closed lane is a documented result, not a reason to stop (Section 8).

7 Limitations

- **No floor above Tawan’s published value.** Every number certified by the program in this dispute—0.6735471309049393, 0.6734729658195391, 0.67296645387858, 0.6730965989022086—was retired, and the mass-admissible F_T candidates certified to date ($\text{CHAIN_BOUND} = 0.67306445176945029$) sit *below* Tawan’s published 0.6731929114731422 [PROVEN]. Devine’s reported 0.673399 is unaudited. All program-certified floors at the time of writing are below the Tawan floor; the honest statement is that the program’s own certified contribution is currently < 0.6731929 , and the Tawan floor stands.
- **Zero RH evidence.** Proportion bounds of the form $N_0(T)/N(T) \geq c$ are evidence about the proportion of zeros on the critical line, not about the Riemann hypothesis. Neither the certified numbers nor the density-one theorem move the RH question, in either direction [PROVEN].

- **Inconclusive reproduction of the audit’s decimals.** The exact decimal values $F_V = 0.0081469966113\dots$ and $F_T = 0.0077854336862\dots$ from Devine’s clean-room witness could not be re-derived from the program’s surviving artifacts because the certified run’s environment was not recorded; the verdict ($F_T < 0.0079$ at the witness) is robust across every candidate parameter reading tested [INCONCLUSIVE on the exact decimals; PROVEN on the inequality across readings].
- **Independence gap.** The auditor’s clean-room witness script (`experiments/vstalin_claim_arb_witness.` was not obtained or located on the program’s machines, so the corrected pipeline has been validated internally (two verifiers, two implementations, adversarial audits) but not by the third party that initiated the dispute [INCONCLUSIVE as independence]. TODO{verify}: public ask for the script and pinned commit hashes is outstanding.
- **Density-one residual risk.** The layer-(b) analytic theorem (order-uniform transport of trace moments) is graded certified-candidate, pre-Lean, and unformalized; the Lean layer grinds fed-in constants; the capstone is not artifact-gated. None of this is a proven defect; all of it is unverified-by-mechanical-proof [CONJECTURED as sound, recorded as risk].
- **Effectivity.** The density-one constants are ineffective; the program’s own certified floors are effective but currently below the Tawan floor.

8 Conclusion

The arc of this report is a single curve: an external auditor showed that the program’s best certified number bounded the wrong functional; the program confirmed the audit on three independent lanes, retired its record and every record on the same code path, re-certified under the correct functional, then found—through its own guardrails, before publication—that even the corrected runs violated a theorem-level mass condition, and retired those too. Along the way it surfaced a third certificate defect (the dormant LDL test in its Rust verifier) that had never fired, and it rebuilt its infrastructure so that each bug class is now unrepresentable: mass assertions on the coefficient sets, a sound Gershgorin positivity certificate, self-describing runs, and a shared driver contract across two independent verifier implementations.

Three properties of this event are worth stating as a case study in machine self-audit. First, the *external* audit and the *internal* audit found different things: Devine’s one-page audit caught a functional mismatch (which object is certified); the program’s own re-certification discipline caught a hypothesis violation (which coefficient sets the theorem admits). Second, both defects were invisible to numerical verification: the F_V certificate was numerically perfect, and the mass-violating F_T runs were numerically perfect too; sound arithmetic on the wrong object is the failure mode that interval verification cannot see. Third, the guardrails held: every retired number was retired publicly and labeled, no invalidated claim was defended, and no new floor was published until the theorem’s hypotheses were checked by construction rather than by hope.

The Riemann hypothesis itself is untouched by this report. What this report documents is the system that will, on its next attempt, carry the search forward: a research program whose epistemology was stress-tested by an adversary, and which responded by finding another bug in itself.

Honesty statement

Honesty guardrails of the Riemann program.

1. Never fabricate a proof, lemma, or numerical result. No exception.
2. Every claim is labeled: PROVEN / CHECKED NUMERICALLY / CONJECTURED / ABANDONED.
3. Nothing counts as progress until adversarial validators fail to break it.
4. Never weaken a validator to make a result pass.
5. A wrong, confident result is worse than no result—it poisons the whole search.

This report is written under these guardrails. Every number above is either reproduced from the experiment record and labeled, or marked TODO{verify} where the surviving artifacts no longer determine it.

Reproducibility

All computations referenced in this report were run with interval arithmetic (Arb) or exact rational / high-precision arithmetic (mpmath, python-flint) [3, 4]; verifier sources and logs are in the program repository (research notes of 2026-08-18 through 2026-08-24; commit c6a8f5e for the Gershgorin repair). The auditor’s clean-room witness script was not available to the program (Section 7).

References

- [1] Tawanerguo. *Bellman–coboundary proof of a zeta simple-zeros proportion floor*, repository tawanerguo-zeta-simple-zeros: BELLMAN_COBBOUNDARY_PROOF.md, paper/riemann.tex, tools/verify_coboundary.cpp.
- [2] Joshua. *Zeta density-one reproduction*, repository JoshuaHKU/zeta-density-one-reproduction, HEAD cdc2e41, Zenodo DOI 10.5281/zenodo.22065921: certification/certify84.py, certification/certify91.py, lean/CertificateK14.lean, repro/engines/tt_moments.py, paper.tex.
- [3] F. Johansson. *Arb: a C library for arbitrary-precision ball arithmetic*. <https://arblib.org/>.
- [4] F. Johansson et al. *python-flint: Python bindings for FLINT*. <https://github.com/flintlib/python-flint>.